

Comprehensive Data Analysis Report

From Raw Data to Decisions: Cleaning, Visualization and Predictive Modeling in R

Suruthi D

30 October 2026

Table of Contents

1. Introduction and Objectives 3
1.1 The three projects at a glance 3
2. Methodology and Workflow 3
3. Part A: Data Preparation and Exploratory Analysis *(Week 1)* 4
3.1 Dataset overview 4
3.2 Missing values 6
3.3 Cleaning and imputation 8
3.4 Outlier detection 9
3.5 Transformation, scaling and encoding 11
3.6 Exploratory analysis of the cleaned data 13
4. Part B: Data Visualization and Insight Communication *(Week 2)* 16
4.1 Dataset overview 16
4.2 Choosing the right chart for each question 17
4.3 Visualizations and interpretation 17
4.4 Quantifying what the charts show 21
5. Part C: Statistical Analysis and Predictive Modeling *(Week 3)* 22
5.1 Dataset and rationale 22
5.2 Hidden missing values 23
5.3 Exploratory statistics and hypothesis tests 24
5.4 Modeling approach 27
5.5 Interpreting the logistic regression 28
5.6 Final evaluation on the held-out test set 30
5.7 Strengths and limitations of the model 32
6. Integrated Results 32
7. Discussion and Business Implications 33
8. Challenges Overcome and Lessons Learned 33
9. Recommendations and Future Directions 34
10. Conclusion and Limitations 34
11.Screenshots of the RStudio Workflow 35
Appendix 35

1. Introduction and Objectives

This report consolidates three weeks of work into one document that follows data through its full life cycle in R: from a messy raw table, to clear visual communication, to a validated predictive model. The objectives are to:

1. Show **how data is prepared**: missing values, outliers, scaling and encoding (Week 1, `msleep`).
2. Show **how patterns are communicated**: choosing the right chart for each question (Week 2, `gapminder`).
3. Show **how a model is built and judged**: hypothesis tests, cross-validation, diagnostics and metrics (Week 3, Pima Indians Diabetes).
4. Connect the three stages, explain what the results mean for decision-makers, and record the reasoning behind every decision so the workflow is reproducible.

The report is written for two audiences. Technical readers will find the annotated R code and diagnostics. Non-technical readers can follow the plain-language explanation that introduces each result and the “Hand-off” note that closes each part.

1.1 The three projects at a glance

Overview of the three projects

Stage	Dataset	Core question	Techniques
Week 1: Cleaning and EDA	msleep (ggplot2)	What must messy, mixed-type data go through before it can be trusted?	Missing values, IQR outliers, log/z-score/min-max, one-hot/ordinal/frequency encoding, correlations
Week 2: Visualization	gapminder	How have health and wealth changed, and where are the anomalies?	Histogram, scatter, line and bar charts; log scales; population weighting
Week 3: Modeling	Pima Indians Diabetes	Can routine clinical measurements predict diabetes?	Normality checks, t/Wilcoxon/chi-square tests, logistic regression, 10-fold CV, ROC

2. Methodology and Workflow

All three projects followed the same six-phase workflow. Using one process made the projects comparable and each one easy to audit.

Phase	What happened
1. Understand	Read documentation, inspect with str(), summary(), glimpse()
2. Prepare	Fix missing values, treat outliers, transform and encode variables
3. Explore	Descriptive statistics, correlations and charts
4. Model	Test hypotheses, fit a model, validate with cross-validation
5. Communicate	Interpret in plain language and state the business impact
6. Share	Version-control the code and outputs on GitHub

Reproducibility. Everything is written in R Markdown so that code, output and narrative live in one file. A fixed seed (`set.seed(123)`) is set wherever randomness is involved (data splitting and cross-validation), so a reviewer who reruns the file gets the same numbers. Repository links are in Appendix A.

3. Part A: Data Preparation and Exploratory Analysis *(Week 1)*

3.1 Dataset overview

The **msleep** dataset ships with the `ggplot2` package. It is an updated version of the mammal sleep data of Allison and Cicchetti (1976) and holds sleep patterns, brain weight and body weight for 83 mammal species. It was chosen because it has missing values in both numeric and categorical columns, a mix of variable types, and strong outliers and skew (elephants versus mice), which makes normalisation and outlier handling meaningful rather than cosmetic.

```
data("msleep", package = "ggplot2")
ms_raw <- msleep          # untouched copy for before/after comparisons
dim(ms_raw)

## [1] 83 11

ms_dictionary <- tibble(
  Variable = c("name", "genus", "vore", "order", "conservation",
               "sleep_total", "sleep_rem", "sleep_cycle", "awake",
               "brainwt", "bodywt"),
  Type = c(rep("Categorical", 5), rep("Numeric", 6)),
  Description = c("Common name of the animal", "Genus", "Diet: carni, herbi,
insecti, omni",
                  "Taxonomic order", "Conservation status (IUCN-style code)",
                  "Total sleep per day (hours)", "REM sleep per day (hours)",
                  "Length of sleep cycle (hours)", "Hours awake per day",
```

```
                  "Brain weight (kg)", "Body weight (kg)"))
kable(ms_dictionary, caption = "Data dictionary: msleep")
```

Data dictionary: msleep

Variable	Type	Description
name	Categorical	Common name of the animal
genus	Categorical	Genus
vore	Categorical	Diet: carni, herbi, insecti, omni
order	Categorical	Taxonomic order
conservation	Categorical	Conservation status (IUCN-style code)
sleep_total	Numeric	Total sleep per day (hours)
sleep_rem	Numeric	REM sleep per day (hours)
sleep_cycle	Numeric	Length of sleep cycle (hours)
awake	Numeric	Hours awake per day
brainwt	Numeric	Brain weight (kg)
bodywt	Numeric	Body weight (kg)

```
str(ms_raw)
```

```
## tibble [83 × 11] (S3: tbl_df/tbl/data.frame)
##  $ name        : chr [1:83] "Cheetah" "Owl monkey" "Mountain beaver" "Grea
ter short-tailed shrew" ...
##  $ genus       : chr [1:83] "Acinonyx" "Aotus" "Aplodontia" "Blarina" ...
##  $ vore        : chr [1:83] "carni" "omni" "herbi" "omni" ...
##  $ order       : chr [1:83] "Carnivora" "Primates" "Rodentia" "Soricomorph
a" ...
##  $ conservation: chr [1:83] "lc" NA "nt" "lc" ...
##  $ sleep_total : num [1:83] 12.1 17 14.4 14.9 4 14.4 8.7 7 10.1 3 ...
##  $ sleep_rem   : num [1:83] NA 1.8 2.4 2.3 0.7 2.2 1.4 NA 2.9 NA ...
##  $ sleep_cycle : num [1:83] NA NA NA 0.133 0.667 ...
##  $ awake       : num [1:83] 11.9 7 9.6 9.1 20 9.6 15.3 17 13.9 21 ...
##  $ brainwt     : num [1:83] NA 0.0155 NA 0.00029 0.423 NA NA NA 0.07 0.098
2 ...
##  $ bodywt      : num [1:83] 50 0.48 1.35 0.019 600 ...
```

```
summary(ms_raw)
```

```
##         name           genus           vore           order       conservati
on
##  Length   :83   Length   :83   Length   :83   Length   :83   Length   :83

##  N.unique :83   N.unique :77   N.unique : 4   N.unique :19   N.unique : 6

##  N.blank  : 0   N.blank  : 0   N.blank  : 0   N.blank  : 0   N.blank  : 0

##  Min.nchar: 3   Min.nchar: 3   Min.nchar: 4   Min.nchar: 6   Min.nchar: 2
```

```
##  Max.nchar:30   Max.nchar:13   Max.nchar: 7   Max.nchar:15   Max.nchar:12

##                                NAs      : 7                  NAs      :29

##

##   sleep_total      sleep_rem      sleep_cycle         awake
##  Min.   : 1.90   Min.   :0.100   Min.   :0.1167   Min.   : 4.10
##  1st Qu.: 7.85   1st Qu.:0.900   1st Qu.:0.1833   1st Qu.:10.25
##  Median :10.10   Median :1.500   Median :0.3333   Median :13.90
##  Mean   :10.43   Mean   :1.875   Mean   :0.4396   Mean   :13.57
##  3rd Qu.:13.75   3rd Qu.:2.400   3rd Qu.:0.5792   3rd Qu.:16.15
##  Max.   :19.90   Max.   :6.600   Max.   :1.5000   Max.   :22.10
##                  NAs    :22      NAs    :51
##     brainwt            bodywt
##  Min.   :0.00014   Min.   :   0.005
##  1st Qu.:0.00290   1st Qu.:   0.174
##  Median :0.01240   Median :   1.670
##  Mean   :0.28158   Mean   : 166.136
##  3rd Qu.:0.12550   3rd Qu.:  41.750
##  Max.   :5.71200   Max.   :6654.000
##  NAs    :27
```

3.2 Missing values

Missingness was uneven across variables, so one rule for every column would have been a mistake.

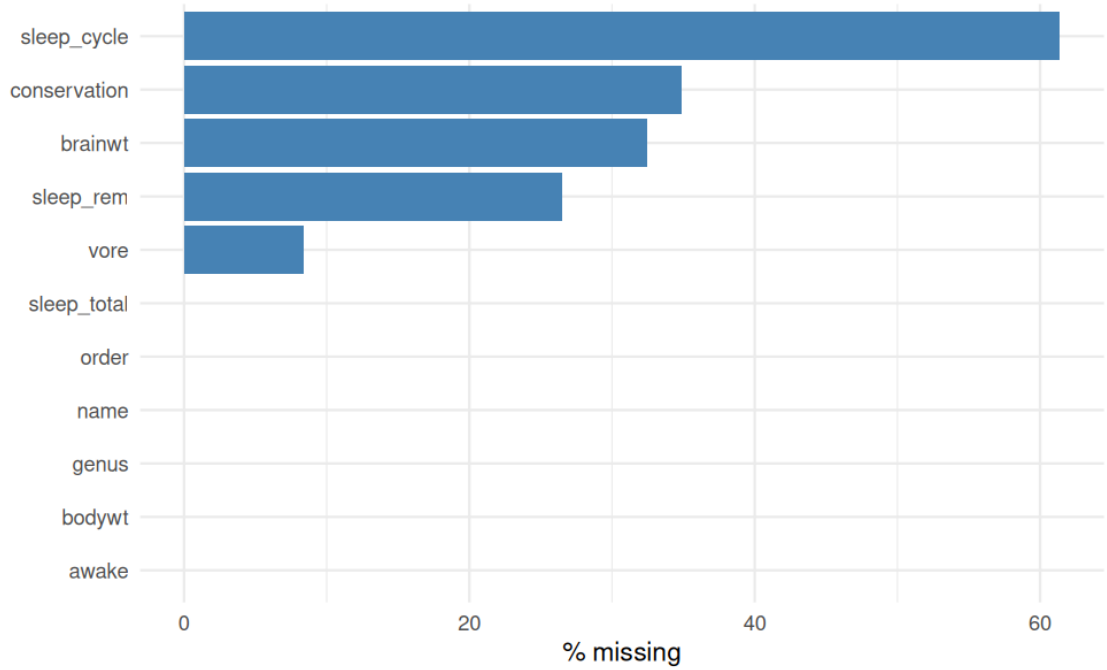
```
ms_miss <- tibble(
  variable    = names(ms_raw),
  n_missing   = colSums(is.na(ms_raw)),
  pct_missing = round(100 * n_missing / nrow(ms_raw), 1)
) %>% arrange(desc(n_missing))
kable(ms_miss, caption = "Missing values per variable in the raw data")
```

Missing values per variable in the raw data

variable	n_missing	pct_missing
sleep_cycle	51	61.4
conservation	29	34.9
brainwt	27	32.5
sleep_rem	22	26.5
vore	7	8.4
name	0	0.0
genus	0	0.0
order	0	0.0
sleep_total	0	0.0

variable	n_missing	pct_missing
awake	0	0.0
bodywt	0	0.0

```
ggplot(ms_miss, aes(x = reorder(variable, pct_missing), y = pct_missing)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(x = NULL, y = "% missing")
```



Percentage of missing values per variable (raw data)

The strategy depends on the variable type and how much is missing:

Situation	Strategy	Reason
Column with more than 50% missing	Drop the column	Imputing most of a column mostly invents data
Missing categorical value	Recode as an explicit "unknown" level	Missingness may itself be informative; avoids deleting rows
Missing numeric value	Median of the animal's diet group (vore), falling back to the overall median	The median is robust to heavy skew; the group median keeps biological structure
Imputed numeric values	Add a logical flag column	Keeps the imputation transparent for later modelling

3.3 Cleaning and imputation

```
ms <- ms_raw %>%
  mutate(across(where(is.character), str_trim)) %>%
  mutate(across(c(order, genus, vore, conservation), as.factor))

# (a) Drop columns with too much missingness
threshold <- 0.5
high_miss <- names(which(colMeans(is.na(ms)) > threshold))
cat("Columns dropped (>", threshold * 100, "% missing):", high_miss, "\n")

## Columns dropped (> 50 % missing): sleep_cycle

ms <- ms %>% select(-all_of(high_miss))

# (b) Categorical NA -> explicit "unknown" level
ms <- ms %>%
  mutate(across(c(vore, conservation),
                ~ factor(replace_na(as.character(.x), "unknown"))))

# (c) Numeric NA -> group median with overall-median fallback
impute_group_median <- function(x, g) {
  out <- ave(x, g, FUN = function(v) { v[is.na(v)] <- median(v, na.rm = TRUE);
 v })
  out[is.na(out)] <- median(x, na.rm = TRUE)
  out
}

ms <- ms %>%
  mutate(brainwt_imputed   = is.na(brainwt),
         sleep_rem_imputed = is.na(sleep_rem),
         brainwt   = impute_group_median(brainwt,   vore),
         sleep_rem = impute_group_median(sleep_rem, vore))

cat("Total remaining NA:", sum(is.na(ms)), "\n")

## Total remaining NA: 0
```

Sensitivity check: imputation should not distort the distributions much.

```
ms_check <- tibble(
  variable    = c("sleep_rem", "brainwt"),
  mean_before = c(mean(ms_raw$sleep_rem, na.rm = TRUE), mean(ms_raw$brainwt,
na.rm = TRUE)),
  mean_after  = c(mean(ms$sleep_rem), mean(ms$brainwt)),
  sd_before   = c(sd(ms_raw$sleep_rem, na.rm = TRUE), sd(ms_raw$brainwt, na.r
m = TRUE)),
  sd_after    = c(sd(ms$sleep_rem), sd(ms$brainwt))
) %>% mutate(across(where(is.numeric), ~ round(.x, 4)))
kable(ms_check, caption = "Mean and SD before and after imputation")
```

Mean and SD before and after imputation

variable	mean_before	mean_after	sd_before	sd_after
sleep_rem	1.8754	1.8102	1.2983	1.1528
brainwt	0.2816	0.1974	0.9764	0.8090

3.4 Outlier detection

Outliers are flagged with the **IQR rule**: a value is an outlier if it lies below Q1 - 1.5 x IQR or above Q3 + 1.5 x IQR.

```
iqr_outliers <- function(x) {
  q   <- quantile(x, c(0.25, 0.75), na.rm = TRUE)
  iqr <- q[2] - q[1]
  x < q[1] - 1.5 * iqr | x > q[2] + 1.5 * iqr
}

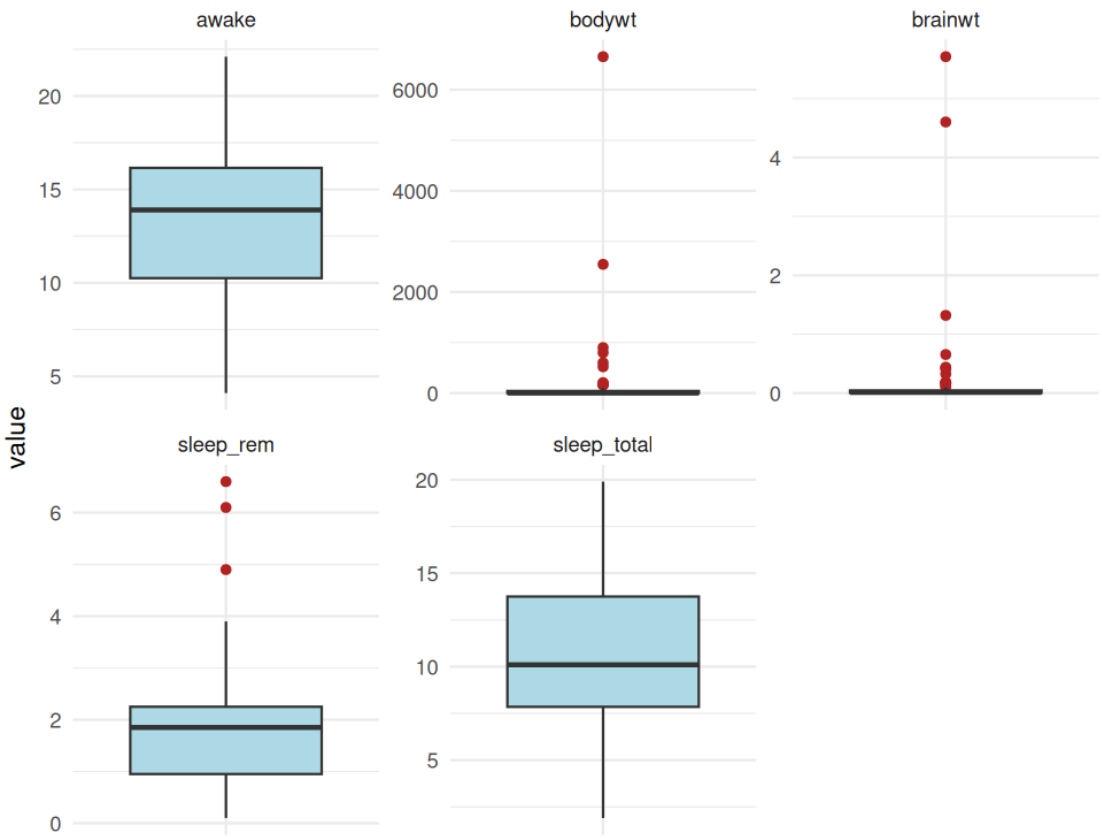
ms_num_vars <- ms %>% select(where(is.numeric)) %>% names()

ms_outliers <- map_dfr(ms_num_vars, function(v) {
  flag <- iqr_outliers(ms[[v]])
  tibble(variable = v, n_outliers = sum(flag), pct_outliers = round(100 * mea
n(flag), 1))
})
kable(ms_outliers, caption = "Outliers per numeric variable (IQR rule)")
```

Outliers per numeric variable (IQR rule)

variable	n_outliers	pct_outliers
sleep_total	0	0.0
sleep_rem	3	3.6
awake	0	0.0
brainwt	16	19.3
bodywt	11	13.3

```
ms %>%
  select(all_of(ms_num_vars)) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "value") %>%
  ggplot(aes(x = variable, y = value)) +
  geom_boxplot(fill = "lightblue", outlier.colour = "firebrick") +
  facet_wrap(~ variable, scales = "free") +
  theme(axis.text.x = element_blank(), axis.title.x = element_blank())
```



Boxplots of numeric variables (free scales) showing outliers

```
ms %>%
  filter(iqr_outliers(bodywt)) %>%
  select(name, bodywt, brainwt) %>%
  arrange(desc(bodywt)) %>%
  kable(caption = "Body-weight outliers")
```

Body-weight outliers

Name	bodywt	brainwt
African elephant	6654.000	5.712000
Asian elephant	2547.000	4.603000
Giraffe	899.995	0.012285
Pilot whale	800.000	0.044500
Cow	600.000	0.423000
Horse	521.000	0.655000
Brazilian tapir	207.501	0.169000
Donkey	187.000	0.419000
Bottle-nosed dolphin	173.330	0.044500
Tiger	162.564	0.044500

Name	bodywt	brainwt
Lion	161.499	0.044500

Decision: these are genuine biological measurements (large mammals), not data-entry errors, so deleting them would remove real information. They are flagged, and their influence is reduced with a log transformation instead.

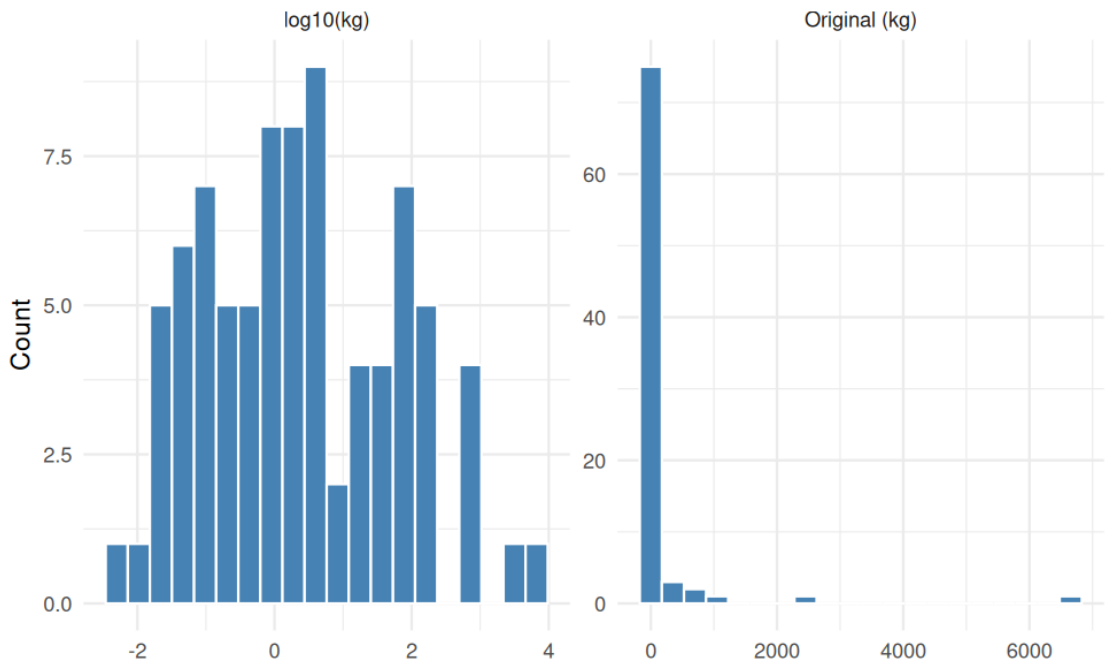
3.5 Transformation, scaling and encoding

```
ms <- ms %>%
  mutate(bodywt_outlier  = iqr_outliers(bodywt),
         brainwt_outlier = iqr_outliers(brainwt),
         log_bodywt  = log10(bodywt),
         log_brainwt = log10(brainwt))

minmax <- function(x) (x - min(x)) / (max(x) - min(x))
scale_vars <- c("sleep_total", "sleep_rem", "awake", "log_brainwt", "log_body
wt")

ms_norm <- ms %>%
  mutate(across(all_of(scale_vars),
                list(z = ~ as.numeric(scale(.x)), mm = minmax)))

bind_rows(tibble(scale = "Original (kg)", value = ms$bodywt),
          tibble(scale = "log10(kg)",     value = ms$log_bodywt)) %>%
  ggplot(aes(value)) +
  geom_histogram(bins = 20, fill = "steelblue", colour = "white") +
  facet_wrap(~ scale, scales = "free") +
  labs(x = NULL, y = "Count")
```



Body weight before and after log10 transformation

Three different encodings are used, each matched to the variable:

Variable	Encoding	Reason
vore (5 levels, no order)	One-hot (dummy)	No natural order between diets
conservation	Ordinal	Threat categories have a natural order of risk
order (many levels)	Frequency encoding	One-hot would add too many columns for 83 rows

```
onehot <- model.matrix(~ vore - 1, data = ms_norm)
colnames(onehot) <- paste0("vore_", levels(ms_norm$vore))

# "domesticated" and "unknown" are not points on the threat scale -> NA
threat_order <- c("lc", "cd", "nt", "vu", "en")

ms_enc <- bind_cols(ms_norm, as_tibble(onehot)) %>%
  mutate(conservation_ord = match(as.character(conservation), threat_order),
         order_freq = ave(rep(1, n()), order, FUN = sum) / n())

ms_enc %>%
  select(name, vore, starts_with("vore_"), conservation, conservation_ord, or
der, order_freq) %>%
  head(6) %>%
  kable(digits = 3, caption = "First rows of the encoded variables")
```

First rows of the encoded variables

name	vore	vore_carni	vore_herbi	vore_insecti	vore_omni	vore_unknown	conservation	conservation_ord	order	order_freq
Cheetah	carni	1	0	0	0	0	lc	1	Carnivora	0.145
Owl monkey	omni	0	0	0	1	0	unknown	NA	Primates	0.145
Mountain beaver	herbi	0	1	0	0	0	nt	3	Rodentia	0.265
Greater short-tailed shrew	omni	0	0	0	1	0	lc	1	Soricomorpha	0.060
Cow	herbi	0	1	0	0	0	domesticated	NA	Artiodactyla	0.072
Three-toed sloth	herbi	0	1	0	0	0	unknown	NA	Pilosa	0.012

```
write_csv(ms_enc, "msleep_clean.csv")
```

3.6 Exploratory analysis of the cleaned data

```
ms_desc <- ms %>%
  select(all_of(ms_num_vars)) %>%
  pivot_longer(everything(), names_to = "variable") %>%
  group_by(variable) %>%
  summarise(n = n(), mean = mean(value), median = median(value), sd = sd(valu
e),
            min = min(value), max = max(value),
            skewness = mean((value - mean(value))^3) / sd(value)^3,
            .groups = "drop") %>%
  mutate(across(where(is.numeric), ~ round(.x, 3)))
kable(ms_desc, caption = "Descriptive statistics of numeric variables")
```

Descriptive statistics of numeric variables

variable	n	mean	median	sd	min	max	skewness
awake	83	13.567	13.900	4.452	4.100	22.100	-0.051
bodywt	83	166.136	1.670	786.840	0.005	6654.000	7.100
brainwt	83	0.197	0.012	0.809	0.000	5.712	5.789
sleep_rem	83	1.810	1.850	1.153	0.100	6.600	1.663
sleep_total	83	10.434	10.100	4.450	1.900	19.900	0.052

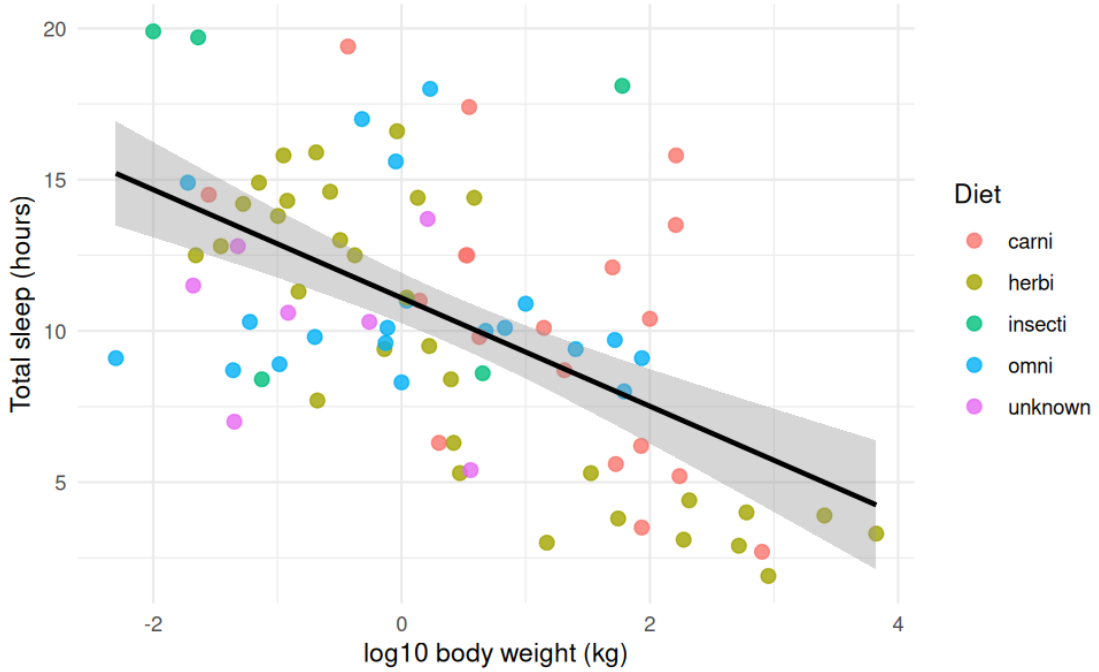
```
ms_by_vore <- ms %>%
  group_by(vore) %>%
  summarise(n = n(),
            mean_sleep   = round(mean(sleep_total), 2),
            median_sleep = round(median(sleep_total), 2),
            mean_rem     = round(mean(sleep_rem), 2),
            .groups = "drop") %>%
```

```
  arrange(desc(mean_sleep))
kable(ms_by_vore, caption = "Sleep by diet group")
```

Sleep by diet group

vore	n	mean_sleep	median_sleep	mean_rem
insecti	5	14.94	18.1	3.42
omni	20	10.93	9.9	1.95
carni	19	10.38	10.4	2.13
unknown	7	10.19	10.6	1.91
herbi	32	9.51	10.3	1.26

```
ggplot(ms, aes(log_bodywt, sleep_total)) +
  geom_point(aes(colour = vore), size = 2.5, alpha = 0.8) +
  geom_smooth(method = "lm", formula = y ~ x, colour = "black") +
  labs(x = "log10 body weight (kg)", y = "Total sleep (hours)", colour = "Die
t")
```

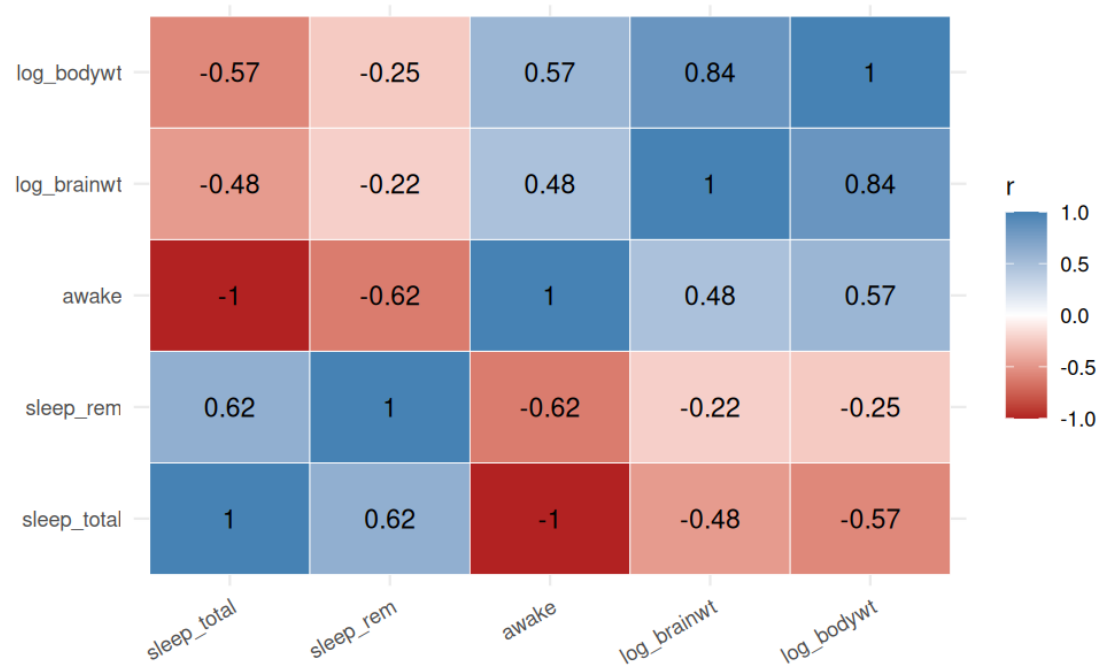


Total sleep versus log10 body weight, coloured by diet

```
cor_vars <- c("sleep_total", "sleep_rem", "awake", "log_brainwt", "log_bodywt
")
ms_cmat <- cor(ms[cor_vars])

as.data.frame(as.table(ms_cmat)) %>%
  ggplot(aes(Var1, Var2, fill = Freq)) +
  geom_tile(colour = "white") +
  geom_text(aes(label = round(Freq, 2))) +
  scale_fill_gradient2(low = "firebrick", mid = "white", high = "steelblue",
```

```
limits = c(-1, 1)) +
  labs(x = NULL, y = NULL, fill = "r") +
  theme(axis.text.x = element_text(angle = 30, hjust = 1))
```



Correlation heatmap of the msleep numeric variables

Findings from Part A.

- The raw data had 136 missing cells; 0 remain after cleaning.
- Total sleep and log body weight have r = -0.57: larger animals tend to sleep less.
- log brain weight and log body weight are very highly correlated (r = 0.84), reflecting allometric scaling.
- `awake` equals 24 minus `sleep_total` (r = -1), so only one of the two should enter a model.
- The diet group with the highest average sleep is **insecti**, but group sizes are uneven, so this is preliminary.

Hand-off to Part B: a documented, fully populated dataset, and the habit of asking *why* a value is missing or extreme before deciding what to do with it.

4. Part B: Data Visualization and Insight Communication *(Week 2)*

4.1 Dataset overview

The **gapminder** dataset tracks 142 countries every five years from 1952 to 2007, giving 1704 rows. Unlike msleep, it is already clean, so the effort moves from repairing data to choosing the right chart for each question.

```
kable(tibble(
  Variable = c("country", "continent", "year", "lifeExp", "pop", "gdpPercap"),
  Type = c("Categorical", "Categorical", "Numeric", "Numeric", "Numeric", "Nu
meric"),
  Description = c("Country name", "Continent", "Year (every 5 years)",
                  "Life expectancy at birth (years)", "Population",
                  "GDP per capita (inflation-adjusted US$)")),
  caption = "Data dictionary: gapminder")
```

Data dictionary: gapminder

Variable	Type	Description
country	Categorical	Country name
continent	Categorical	Continent
year	Numeric	Year (every 5 years)
lifeExp	Numeric	Life expectancy at birth (years)
pop	Numeric	Population
gdpPercap	Numeric	GDP per capita (inflation-adjusted US$)

```
cat("Missing values:", sum(is.na(gapminder)), "\n")
```

```
## Missing values: 0
```

```
summary(gapminder)
```

```
##         country        continent        year         lifeExp
##  Afghanistan:  12   Africa  :624   Min.   :1952   Min.   :23.60
##  Albania    :  12   Americas:300   1st Qu.:1966   1st Qu.:48.20
##  Algeria    :  12   Asia    :396   Median :1980   Median :60.71
##  Angola     :  12   Europe  :360   Mean   :1980   Mean   :59.47
##  Argentina  :  12   Oceania : 24   3rd Qu.:1993   3rd Qu.:70.85
##  Australia  :  12                  Max.   :2007   Max.   :82.60
##  (Other)    :1632
##       pop              gdpPercap
##  Min.   :6.001e+04   Min.   :   241.2
##  1st Qu.:2.794e+06   1st Qu.:  1202.1
##  Median :7.024e+06   Median :  3531.8
##  Mean   :2.960e+07   Mean   :  7215.3
##  3rd Qu.:1.959e+07   3rd Qu.:  9325.5
```

```
##  Max.   :1.319e+09   Max.   :113523.1
##
```

```
gap07 <- gapminder %>% filter(year == 2007)

gap07 %>%
  group_by(continent) %>%
  summarise(countries = n(),
            mean_life  = round(mean(lifeExp), 1),
            median_gdp = round(median(gdpPercap)),
            pop_millions = round(sum(as.numeric(pop)) / 1e6),
            .groups = "drop") %>%
  kable(caption = "2007 snapshot by continent")
```

2007 snapshot by continent

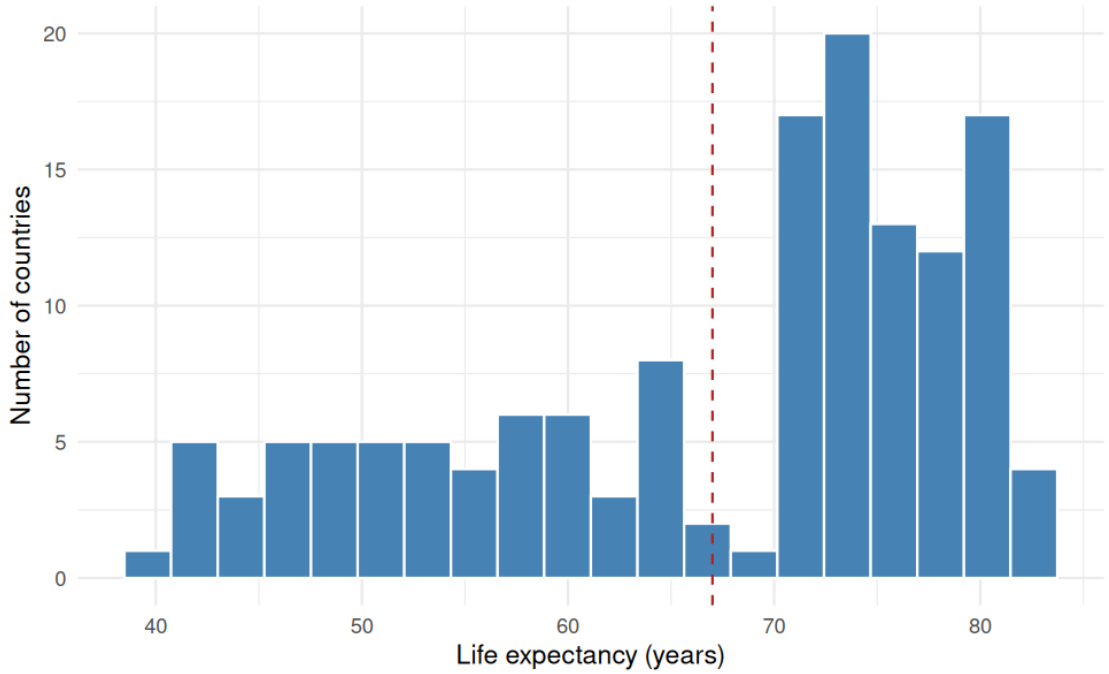
continent	countries	mean_life	median_gdp	pop_millions
Africa	52	54.8	1452	930
Americas	25	73.6	8948	899
Asia	33	70.7	4471	3812
Europe	30	77.6	28054	586
Oceania	2	80.7	29810	25

4.2 Choosing the right chart for each question

Chart	Question it answers	Why it fits
Histogram	How is life expectancy distributed worldwide in 2007?	Shows shape, spread and any split into groups
Scatter (log x)	Does wealth relate to life expectancy?	Two continuous variables; the log scale handles the wide income range
Line	How has life expectancy changed over time by continent?	Time is the natural x-axis; lines show trend and gaps
Bar	Which continents have the highest GDP per capita?	Compares one summary value across categories

4.3 Visualizations and interpretation

```
ggplot(gap07, aes(lifeExp)) +
  geom_histogram(bins = 20, fill = "steelblue", colour = "white") +
  geom_vline(aes(xintercept = mean(lifeExp)), colour = "firebrick", linetype
= "dashed") +
  labs(x = "Life expectancy (years)", y = "Number of countries")
```



Distribution of life expectancy across countries, 2007

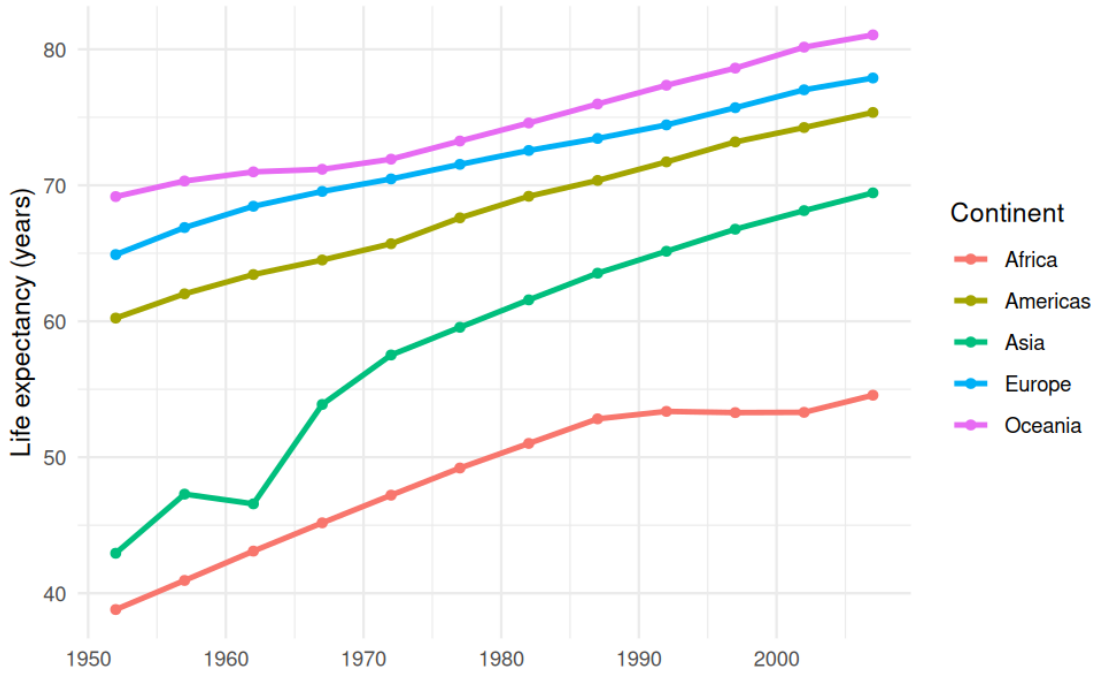
```
ggplot(gap07, aes(gdpPercap, lifeExp, colour = continent, size = pop)) +
  geom_point(alpha = 0.7) +
  scale_x_log10(labels = scales::dollar) +
  scale_size(range = c(1, 12), guide = "none") +
  labs(x = "GDP per capita (log scale)", y = "Life expectancy (years)", colou
r = "Continent")
```



Life expectancy versus GDP per capita (log scale), 2007

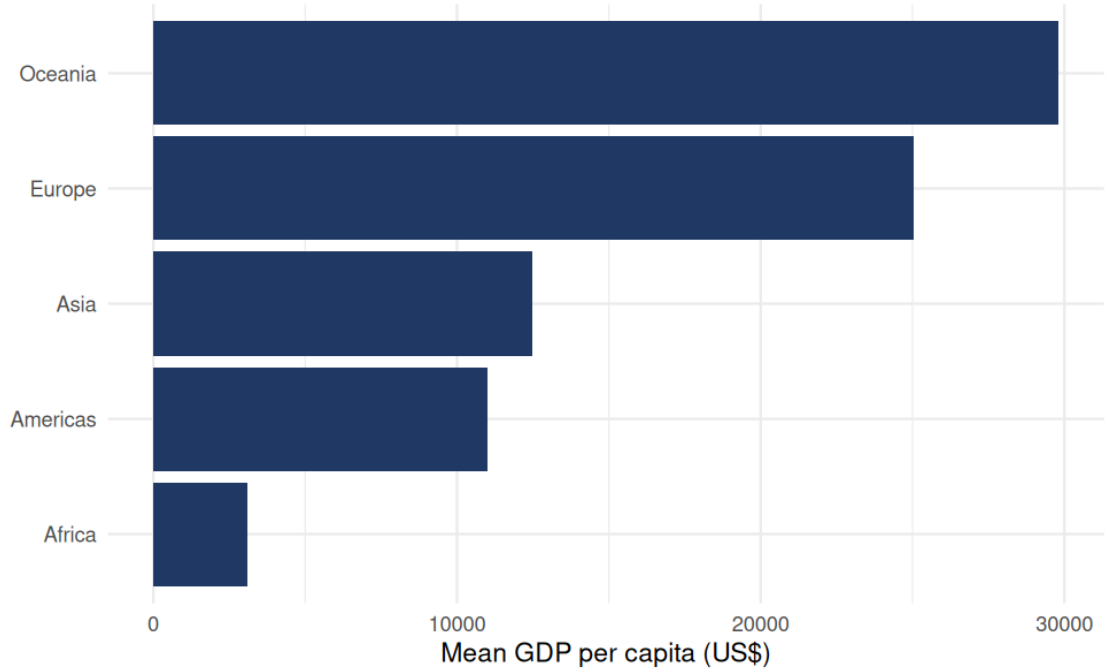
```
life_cont <- gapminder %>%
  group_by(continent, year) %>%
  summarise(life = weighted.mean(lifeExp, pop), .groups = "drop")

ggplot(life_cont, aes(year, life, colour = continent)) +
  geom_line(linewidth = 1.1) + geom_point() +
  labs(x = NULL, y = "Life expectancy (years)", colour = "Continent")
```



Population-weighted life expectancy by continent, 1952 to 2007

```
gap07 %>%
  group_by(continent) %>%
  summarise(gdp = mean(gdpPercap), .groups = "drop") %>%
  ggplot(aes(reorder(continent, gdp), gdp)) +
  geom_col(fill = "#1F3864") +
  coord_flip() +
  labs(x = NULL, y = "Mean GDP per capita (US$)")
```



Mean GDP per capita by continent, 2007

4.4 Quantifying what the charts show

```
gains <- gapminder %>%
  filter(year %in% c(1952, 2007)) %>%
  group_by(continent, year) %>%
  summarise(life = weighted.mean(lifeExp, pop), .groups = "drop") %>%
  pivot_wider(names_from = year, values_from = life, names_prefix = "y") %>%
  mutate(gain = y2007 - y1952) %>%
  arrange(desc(gain)) %>%
  mutate(across(where(is.numeric), ~ round(.x, 1)))
kable(gains, caption = "Life expectancy gain by continent, 1952 to 2007")
```

Life expectancy gain by continent, 1952 to 2007

continent	y1952	y2007	gain
Asia	42.9	69.4	26.5
Africa	38.8	54.6	15.8
Americas	60.2	75.4	15.1
Europe	64.9	77.9	13.0
Oceania	69.2	81.1	11.9

```
changes <- gapminder %>%
  arrange(country, year) %>%
  group_by(country) %>%
  mutate(change = lifeExp - lag(lifeExp)) %>%
  ungroup() %>%
  filter(!is.na(change))
```

```
worst <- changes %>% arrange(change) %>% slice(1)
```

Findings from Part B.

- **Wealth and health.** log10 GDP per capita and life expectancy have r = 0.81 in 2007, and income explains about 65% of the variation in life expectancy across countries. The curve flattens at high incomes: early gains in income are associated with the biggest health gains.
- **Progress over time.** Every continent improved. The largest gain was in **Asia** (26.5 years) and the smallest in **Oceania** (11.9 years).
- **Anomalies.** Despite the global trend, 102 country-periods show a decline. The largest single drop was **Rwanda**, which lost 20.4 years between 1987 and 1992. Averages hide events like this, which is why grouped and country-level views matter.
- **Income does not tell the whole story.** Countries with similar GDP per capita can differ by 10 or more years of life expectancy, which points to factors such as health policy and disease burden.

Hand-off to Part C: visual evidence that relationships can be non-linear, that averages conceal regional differences, and that distributions should be checked before any model assumes they are well behaved.

5. Part C: Statistical Analysis and Predictive Modeling *(Week 3)*

5.1 Dataset and rationale

The **Pima Indians Diabetes** dataset has 768 female patients and eight clinical predictors. It suits predictive modeling because the outcome is binary, the predictors are clinically meaningful, and the sample is large enough for cross-validation yet small enough to interpret.

```
pima <- pima_raw
pima$diabetes <- factor(pima$diabetes, levels = c("pos", "neg"))   # "pos" fi
rst = event of interest

kable(tibble(
  Variable = names(pima),
  Description = c("Number of pregnancies", "Plasma glucose concentration",
                  "Diastolic blood pressure (mm Hg)", "Triceps skin-fold thic
kness (mm)",
                  "2-hour serum insulin (mu U/ml)", "Body mass index",
                  "Diabetes pedigree function", "Age (years)",
                  "Outcome: pos = diabetes, neg = no diabetes")),
  caption = "Data dictionary: Pima Indians Diabetes")
```

Data dictionary: Pima Indians Diabetes

Variable	Description
pregnant	Number of pregnancies
glucose	Plasma glucose concentration
pressure	Diastolic blood pressure (mm Hg)
triceps	Triceps skin-fold thickness (mm)
insulin	2-hour serum insulin (mu U/ml)
mass	Body mass index
pedigree	Diabetes pedigree function
age	Age (years)
diabetes	Outcome: pos = diabetes, neg = no diabetes

```
pima_balance <- tibble(class = names(table(pima$diabetes)),
                       n = as.integer(table(pima$diabetes)),
                       pct = round(100 * prop.table(table(pima$diabetes)), 1))
kable(pima_balance, caption = "Class balance of the outcome")
```

Class balance of the outcome

class	n	pct
pos	268	34.9
neg	500	65.1

5.2 Hidden missing values

A zero is physiologically impossible for glucose, blood pressure, skin-fold thickness, insulin and BMI. These zeros are **disguised missing values**, and the Week 1 lesson applies directly: ask what a value means before using it.

```
cols <- c("glucose", "pressure", "triceps", "insulin", "mass")

zero_tbl <- tibble(variable = cols,
                   n_zero = sapply(pima[cols], function(x) sum(x == 0)),
                   pct_zero = round(100 * n_zero / nrow(pima), 1)) %>%
  arrange(desc(n_zero))
kable(zero_tbl, caption = "Impossible zero values per variable")
```

Impossible zero values per variable

variable	n_zero	pct_zero
insulin	374	48.7
triceps	227	29.6
pressure	35	4.6
mass	11	1.4
glucose	5	0.7

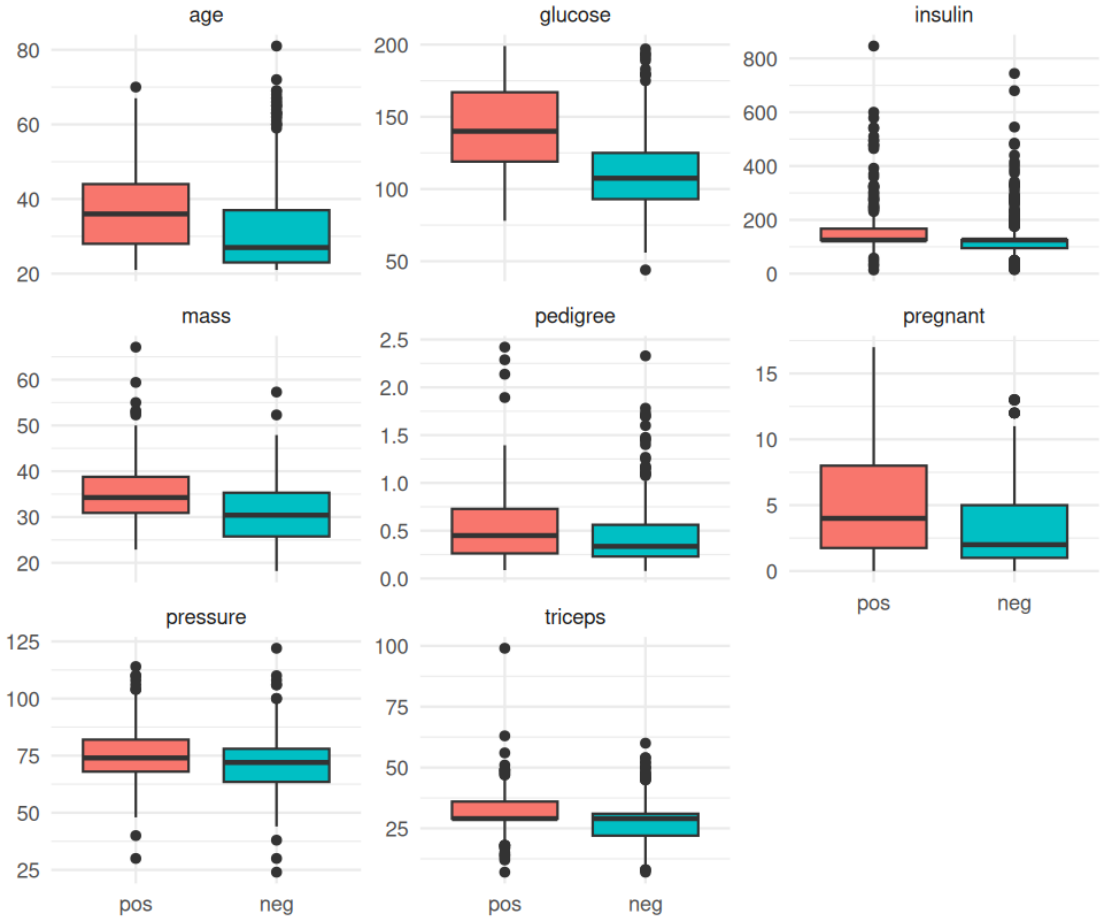
Situation	Strategy	Reason
Impossible zero in a clinical measurement	Recode as NA	The value is missing, not zero
Remaining NA	Replace with the variable median	Robust to the skew seen in insulin and triceps

```
pima[cols] <- lapply(pima[cols], function(x) { x[x == 0] <- NA; x })
pima[cols] <- lapply(pima[cols], function(x) { x[is.na(x)] <- median(x, na.rm
 = TRUE); x })
cat("Total remaining NA:", sum(is.na(pima)), "\n")

## Total remaining NA: 0
```

5.3 Exploratory statistics and hypothesis tests

```
pima %>%
  pivot_longer(-diabetes, names_to = "variable", values_to = "value") %>%
  ggplot(aes(diabetes, value, fill = diabetes)) +
  geom_boxplot(show.legend = FALSE) +
  facet_wrap(~ variable, scales = "free_y") +
  labs(x = NULL, y = NULL)
```



Predictors by diabetes status

Shapiro-Wilk tests check whether each predictor is normal, and non-parametric Wilcoxon tests are used so the conclusions do not depend on that assumption. Cohen's d reports the *size* of the difference, not just whether it is detectable.

```
cohens_d <- function(x, g) {
  x1 <- x[g == "pos"]; x0 <- x[g == "neg"]
  sp <- sqrt(((length(x1) - 1) * var(x1) + (length(x0) - 1) * var(x0)) /
               (length(x1) + length(x0) - 2))
  (mean(x1) - mean(x0)) / sp
}

predictors <- setdiff(names(pima), "diabetes")

pima_tests <- map_dfr(predictors, function(v) {
  tibble(variable  = v,
         mean_pos  = round(mean(pima[[v]][pima$diabetes == "pos"]), 2),
         mean_neg  = round(mean(pima[[v]][pima$diabetes == "neg"]), 2),
         shapiro_p = format.pval(shapiro.test(pima[[v]])$p.value, digits = 3,
 eps = 1e-4),
         wilcox_p  = format.pval(wilcox.test(pima[[v]] ~ pima$diabetes)$p.val
```

```
ue, digits = 3, eps = 1e-4),
         cohens_d  = round(cohens_d(pima[[v]], pima$diabetes), 2))
}) %>% arrange(desc(abs(cohens_d)))

kable(pima_tests, caption = "Group comparison by diabetes status (sorted by e
ffect size)")
```

Group comparison by diabetes status (sorted by effect size)

variable	mean_pos	mean_neg	shapiro_p	wilcox_p	cohens_d
glucose	142.13	110.68	<1e-04	<1e-04	1.19
mass	35.38	30.89	<1e-04	<1e-04	0.69
age	37.07	31.19	<1e-04	<1e-04	0.51
pregnant	4.87	3.30	<1e-04	<1e-04	0.48
triceps	31.69	27.73	<1e-04	<1e-04	0.46
insulin	164.70	127.79	<1e-04	<1e-04	0.44
pedigree	0.55	0.43	<1e-04	<1e-04	0.37
pressure	75.12	70.92	<1e-04	<1e-04	0.35

```
t.test(glucose ~ diabetes, data = pima)

##
##  Welch Two Sample t-test
##
## data:  glucose by diabetes
## t = 14.853, df = 469.08, p-value < 2.2e-16
## alternative hypothesis: true difference in means between group pos and gro
up neg is not equal to 0
## 95 percent confidence interval:
##  27.28789 35.60931
## sample estimates:
## mean in group pos mean in group neg
##          142.1306          110.6820

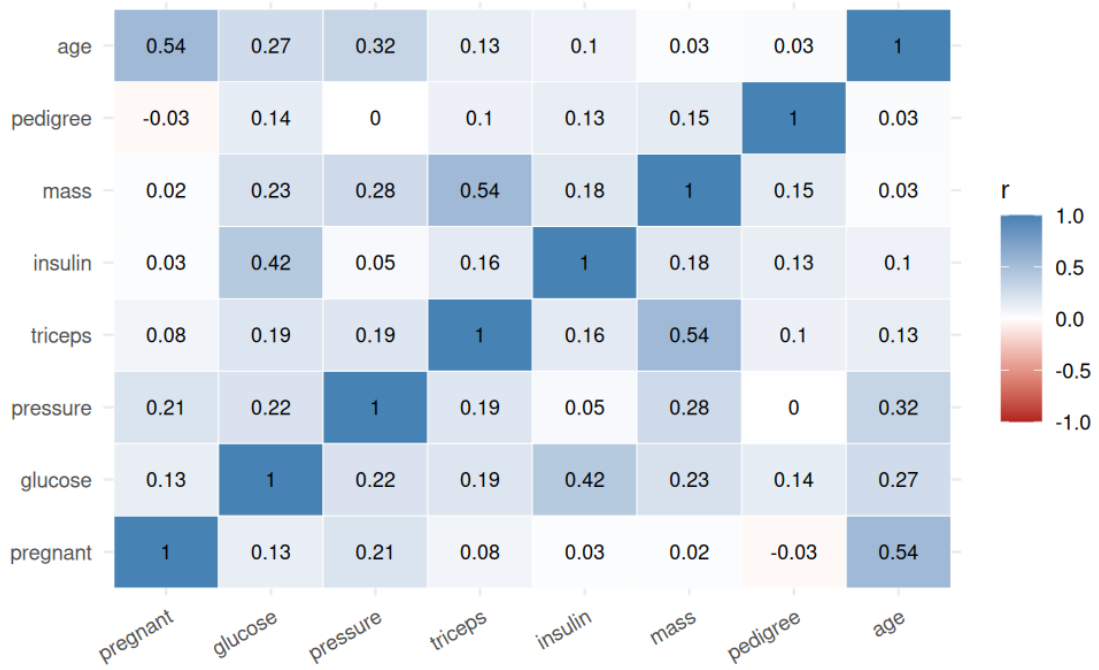
age_group <- cut(pima$age, c(20, 30, 40, 50, 100), include.lowest = TRUE)
chisq.test(table(age_group, pima$diabetes))

##
##  Pearson's Chi-squared test
##
## data:  table(age_group, pima$diabetes)
## X-squared = 73.809, df = 3, p-value = 6.522e-16
```

A p-value below 0.05 means the observed difference would be unlikely if the two groups were really the same. A tiny p-value alone does not make an effect important, which is why the effect size column matters.

```
pima_cmat <- cor(pima[predictors])
```

```r
as.data.frame(as.table(pima_cmat)) %>%
  ggplot(aes(Var1, Var2, fill = Freq)) +
  geom_tile(colour = "white") +
  geom_text(aes(label = round(Freq, 2)), size = 3) +
  scale_fill_gradient2(low = "firebrick", mid = "white", high = "steelblue",
limits = c(-1, 1)) +
  labs(x = NULL, y = NULL, fill = "r") +
  theme(axis.text.x = element_text(angle = 30, hjust = 1))
```



Correlation among predictors

5.4 Modeling approach

Decision	Choice	Reason
Validation	80/20 train-test split, plus 10-fold cross-validation on the training set	The test set stays untouched until the final check; CV gives a stable estimate while tuning
Baseline model	Logistic regression	Interpretable: coefficients become odds ratios
Comparison model	Decision tree (rpart)	Can capture non-linear relationships that logistic regression cannot
Selection metric	ROC AUC	Not distorted by the 65/35 class imbalance
Scaling	Centre and scale inside the CV loop	Odds ratios become “per 1 SD increase”, comparable across predictors

```r
set.seed(123)
idx   <- createDataPartition(pima$diabetes, p = 0.8, list = FALSE)
train <- pima[idx, ]
test  <- pima[-idx, ]
cat("Training rows:", nrow(train), "| Test rows:", nrow(test), "\n")

## Training rows: 615 | Test rows: 153

ctrl <- trainControl(method = "cv", number = 10, classProbs = TRUE,
                     summaryFunction = twoClassSummary, savePredictions = "fi
nal")

set.seed(123)
fit_glm <- train(diabetes ~ ., data = train, method = "glm", family = "binomi
al",
                 metric = "ROC", trControl = ctrl, preProcess = c("center", "
scale"))

set.seed(123)
fit_tree <- train(diabetes ~ ., data = train, method = "rpart",
                  metric = "ROC", trControl = ctrl, tuneLength = 5)

cv_tbl <- bind_rows(
  fit_glm$results  %>% transmute(Model = "Logistic regression", ROC, Sens, Sp
ec),
  fit_tree$results %>% slice_max(ROC, n = 1, with_ties = FALSE) %>%
    transmute(Model = "Decision tree (rpart)", ROC, Sens, Spec)
) %>% mutate(across(where(is.numeric), ~ round(.x, 3)))
kable(cv_tbl, caption = "10-fold cross-validation results (training data)")
```

10-fold cross-validation results (training data)

Model	ROC	Sens	Spec
Logistic regression	0.842	0.577	0.875
Decision tree (rpart)	0.790	0.614	0.812

5.5 Interpreting the logistic regression

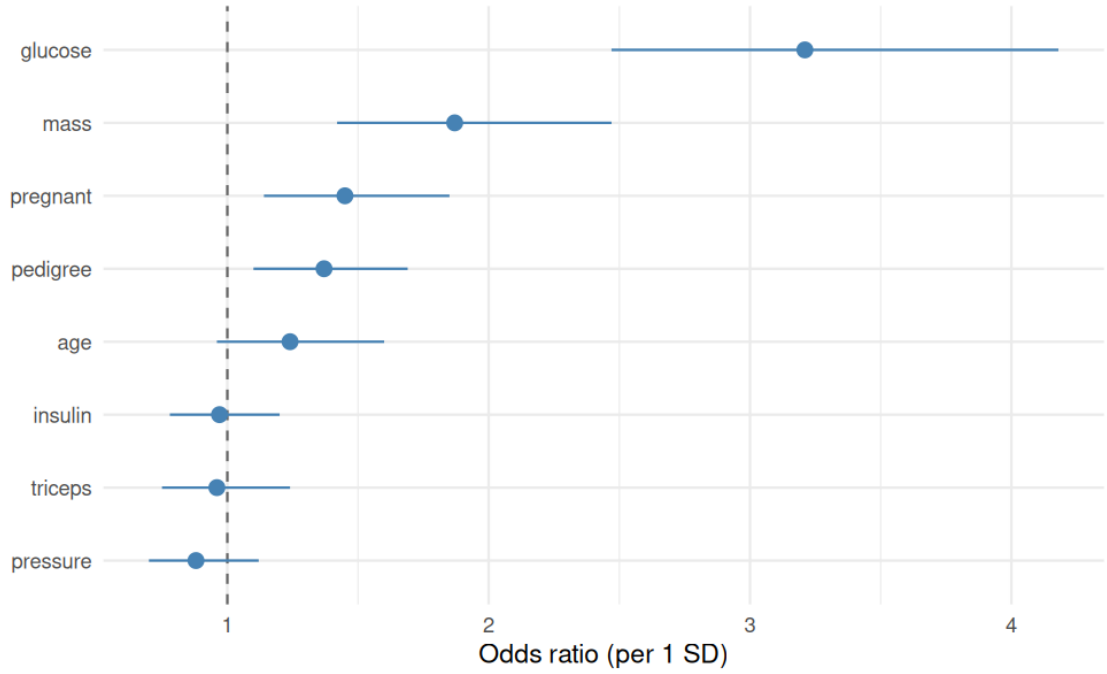
```r
m <- fit_glm$finalModel
or_tbl <- tibble(
  predictor = names(coef(m))[-1],
  odds_ratio = exp(-coef(m))[-1],
  ci_low  = exp(-confint.default(m))[-1, 2],
  ci_high = exp(-confint.default(m))[-1, 1],
  p_value = summary(m)$coefficients[-1, 4]
) %>%
  arrange(desc(odds_ratio)) %>%
  mutate(across(c(odds_ratio, ci_low, ci_high), ~ round(.x, 2)),
         p_value = format.pval(p_value, digits = 3, eps = 1e-4))
```

```
kable(or_tbl, caption = "Odds ratios per 1 SD increase in each predictor (95%
 Wald CI)")
```

Odds ratios of diabetes per 1 SD increase in each predictor (95% Wald CI)

predictor	odds_ratio	ci_low	ci_high	p_value
glucose	3.21	2.47	4.18	< 1e-04
mass	1.87	1.42	2.47	< 1e-04
pregnant	1.45	1.14	1.85	0.00223
pedigree	1.37	1.10	1.69	0.00428
age	1.24	0.96	1.60	0.09318
insulin	0.97	0.78	1.20	0.78242
triceps	0.96	0.75	1.24	0.77317
pressure	0.88	0.70	1.12	0.29299

```
ggplot(or_tbl, aes(odds_ratio, reorder(predictor, odds_ratio))) +
  geom_vline(xintercept = 1, linetype = "dashed", colour = "grey40") +
  geom_pointrange(aes(xmin = ci_low, xmax = ci_high), colour = "steelblue") +
  labs(x = "Odds ratio (per 1 SD)", y = NULL)
```



Odds ratios with 95% confidence intervals

An odds ratio above 1 means that a higher value of the predictor raises the odds of diabetes. A confidence interval that excludes 1 indicates a reliable effect.

5.6 Final evaluation on the held-out test set

```
prob <- predict(fit_glm, newdata = test, type = "prob")$pos
pred <- factor(ifelse(prob >= 0.5, "pos", "neg"), levels = c("pos", "neg"))

cm      <- confusionMatrix(pred, test$diabetes, positive = "pos")
roc_obj <- roc(test$diabetes, prob, levels = c("neg", "pos"), direction = "<")
auc_val <- as.numeric(auc(roc_obj))

kable(as.data.frame.matrix(cm$table), caption = "Confusion matrix (rows = pre
dicted, columns = actual)")
```

Confusion matrix (rows = predicted, columns = actual)

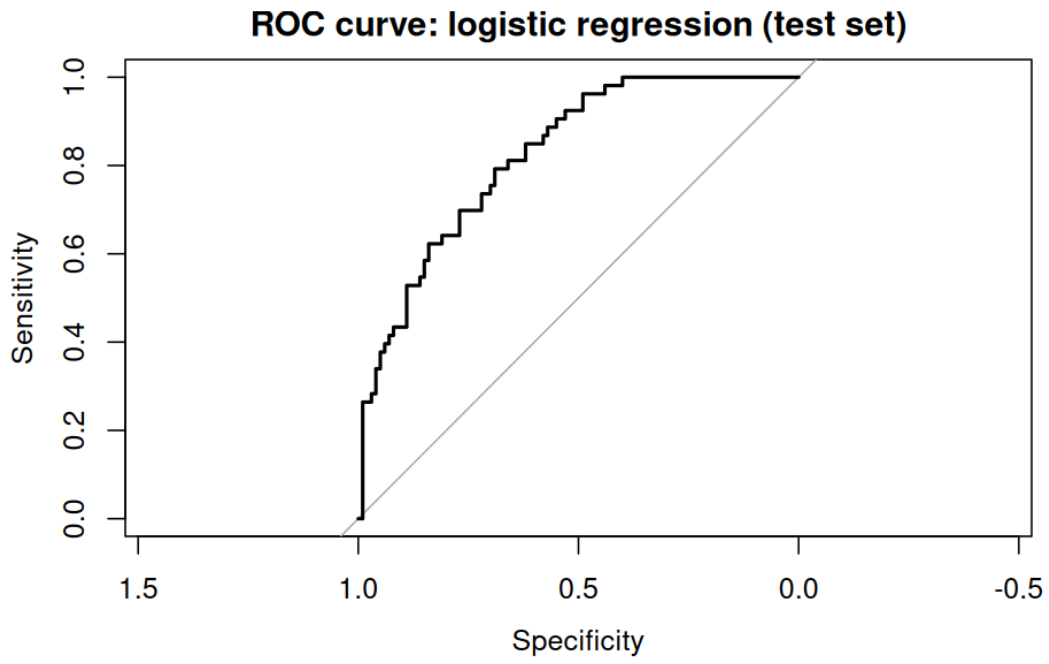
	pos	neg
pos	23	9
neg	30	91

```
kable(tibble(
  Metric = c("Accuracy", "Sensitivity", "Specificity", "Precision", "AUC"),
  Result = round(c(cm$overall["Accuracy"], cm$byClass["Sensitivity"], cm$byCl
ass["Specificity"],
                   cm$byClass["Pos Pred Value"], auc_val), 3),
  Meaning = c("Share of all patients classified correctly",
              "Share of true diabetics the model catches",
              "Share of healthy patients correctly cleared",
              "Share of predicted diabetics who truly have diabetes",
              "Ability to rank a diabetic above a healthy patient (0.5 = chan
ce, 1 = perfect)")),
  caption = "Test-set performance of the logistic regression")
```

Test-set performance of the logistic regression

Metric	Result	Meaning
Accuracy	0.745	Share of all patients classified correctly
Sensitivity	0.434	Share of true diabetics the model catches
Specificity	0.910	Share of healthy patients correctly cleared
Precision	0.719	Share of predicted diabetics who truly have diabetes
AUC	0.824	Ability to rank a diabetic above a healthy patient (0.5 = chance, 1 = perfect)

```
plot(roc_obj, main = "ROC curve: logistic regression (test set)")
```



ROC curve on the test set

Threshold tuning. The default cut-off of 0.5 treats both kinds of error equally. In screening, missing a true case is usually costlier than a false alarm, so a lower threshold may be preferable.

```
thr_tbl <- map_dfr(c(0.3, 0.4, 0.5), function(t) {
  p  <- factor(ifelse(prob >= t, "pos", "neg"), levels = c("pos", "neg"))
  cc <- confusionMatrix(p, test$diabetes, positive = "pos")
  tibble(threshold   = t,
         accuracy    = round(as.numeric(cc$overall["Accuracy"]), 3),
         sensitivity = round(as.numeric(cc$byClass["Sensitivity"]), 3),
         specificity = round(as.numeric(cc$byClass["Specificity"]), 3))
})
kable(thr_tbl, caption = "Effect of the classification threshold")
```

Effect of the classification threshold

threshold	accuracy	sensitivity	specificity
0.3	0.745	0.698	0.77
0.4	0.752	0.585	0.84
0.5	0.745	0.434	0.91

Findings from Part C.

- **Class balance.** 268 of 768 patients (34.9%) have diabetes, so accuracy alone would flatter a model that favours the majority class.
- **Strongest group difference. glucose** shows the largest effect size between diabetic and non-diabetic patients (Cohen's d = 1.19).

- **Strongest model predictor. glucose** has the highest odds ratio per 1 SD increase (3.21).
- **Model comparison.** In cross-validation, **Logistic regression** achieved the higher AUC, so the simpler interpretable model is not necessarily the weaker one.
- **Test-set result.** The logistic regression reached an AUC of 0.824, sensitivity of 0.434 and specificity of 0.91 on data it had never seen.

5.7 Strengths and limitations of the model

- **Strengths:** transparent, fast, and its coefficients agree with clinical expectation (glucose and BMI as main risk signals).
- **Limitations:** it assumes a linear relationship between predictors and the log-odds; median imputation was applied before splitting the data, which lets a small amount of information leak from the test set; and the data comes from one specific population, so the model should not be applied elsewhere without external validation.

6. Integrated Results

The three projects are not independent exercises. Each one depended on the one before it.

Theme	Where it appeared	What it showed
Data quality drives results	Week 1 missing values; Week 3 hidden zeros	The same discipline was needed twice: ask what a value means before trusting it. Unchecked zeros would have distorted every later statistic.
Scale and skew matter	Week 1 log of body weight; Week 2 log of GDP; Week 3 normality checks	Skewed variables hide patterns and break assumptions; transformation revealed structure in all three datasets.
Visuals come before models	Week 2 curved income-health relationship	Looking first revealed non-linearity and motivated checking assumptions and comparing a tree against logistic regression.
Averages hide detail	Week 2 continent trends and country declines; Week 3 class balance	Grouped views and per-class metrics expose what a single summary number conceals.
Effect size beats p-value	Week 1 correlations; Week 3 Cohen's d and odds ratios	Significance says an effect is detectable; effect size says whether it matters.

7. Discussion and Business Implications

- **Health screening.** A simple model built on routine measurements can help prioritise patients for follow-up testing. Its value is triage, helping limited clinical resources reach high-risk people first, not replacing diagnosis. The threshold table shows that the cut-off should be chosen from the cost of a missed case, not left at the default.
- **Policy and investment.** The wealth-health curve suggests that spending has the largest effect where baseline health is lowest, and the country-level declines show that income alone does not protect health outcomes, so targeted health programmes matter.
- **Data governance.** Preparation choices (what to impute, what to drop, what to keep as an outlier) silently shape every conclusion. Documenting them, as done in Part A, makes analyses reviewable and trustworthy.
- **Communication.** The same finding needs different framing for different readers: an odds ratio for a clinician, a trend line for a policy maker, a one-sentence takeaway for a manager.

8. Challenges Overcome and Lessons Learned

Challenge	How it was handled
Deciding how to treat missing data	Treated each variable separately: dropped a column that was mostly empty, group-median imputation for numerics, "unknown" for factors
Skewed and extreme values	Kept genuine extremes, flagged them, and used log transformations instead of deletion
Zeros hiding as real values in Pima	Used domain knowledge to recognise impossible values and recode them as missing
Choosing between models	Compared logistic regression with a decision tree under identical cross-validation folds
Explaining results to non-technical readers	Led each part with the question and closed it with a plain-language takeaway
Environment and reproducibility	Worked in Posit Cloud, fixed seeds, versioned everything on GitHub

Lessons learned

- Cleaning is analysis, not a chore: the most consequential decisions happen before any model is fitted.
- A chart is an argument. Axis, scale and title shape what the reader concludes.
- Accuracy is rarely the right headline metric. The cost of each kind of error should guide the choice.

- Keep the test set untouched until the end, and report effect sizes, not just p-values.
- Documenting the reasoning next to the code makes work reusable.

9. Recommendations and Future Directions

- **Richer models:** compare logistic regression with random forests and gradient boosting, and examine feature importance.
- **Better imputation:** use multiple imputation (e.g. `mice`) and, for the model, impute inside the cross-validation loop to remove leakage.
- **Threshold tuning:** choose the cut-off that matches the real cost of a missed diagnosis.
- **Interactive communication:** build a Shiny dashboard so stakeholders can explore the gapminder trends and model predictions.
- **External validation:** test the diabetes model on a different population before any real-world use.
- **Automation:** wrap the cleaning steps into reusable functions or a pipeline so new data is processed consistently.

10. Conclusion and Limitations

The report followed data through its full journey. In Week 1 messy data was cleaned and made trustworthy; in Week 2 it was made understandable through purposeful visualisation; in Week 3 it was turned into a validated predictive model. The recurring lesson is that a data-driven approach is only as strong as its weakest stage: careful preparation protects the model's validity, clear visuals make findings usable, and honest evaluation prevents overconfidence. A data-driven approach does not remove judgment; it makes judgment visible and testable.

Limitations:

- The three datasets are public, small and unrelated, so the report demonstrates method rather than a single unified business case.
- Median imputation shrinks variance slightly and, for Pima, was applied before the train-test split.
- Correlation and prediction do not establish causation, and species and countries are not fully independent observations.

11.Screenshots of the RStudio Workflow

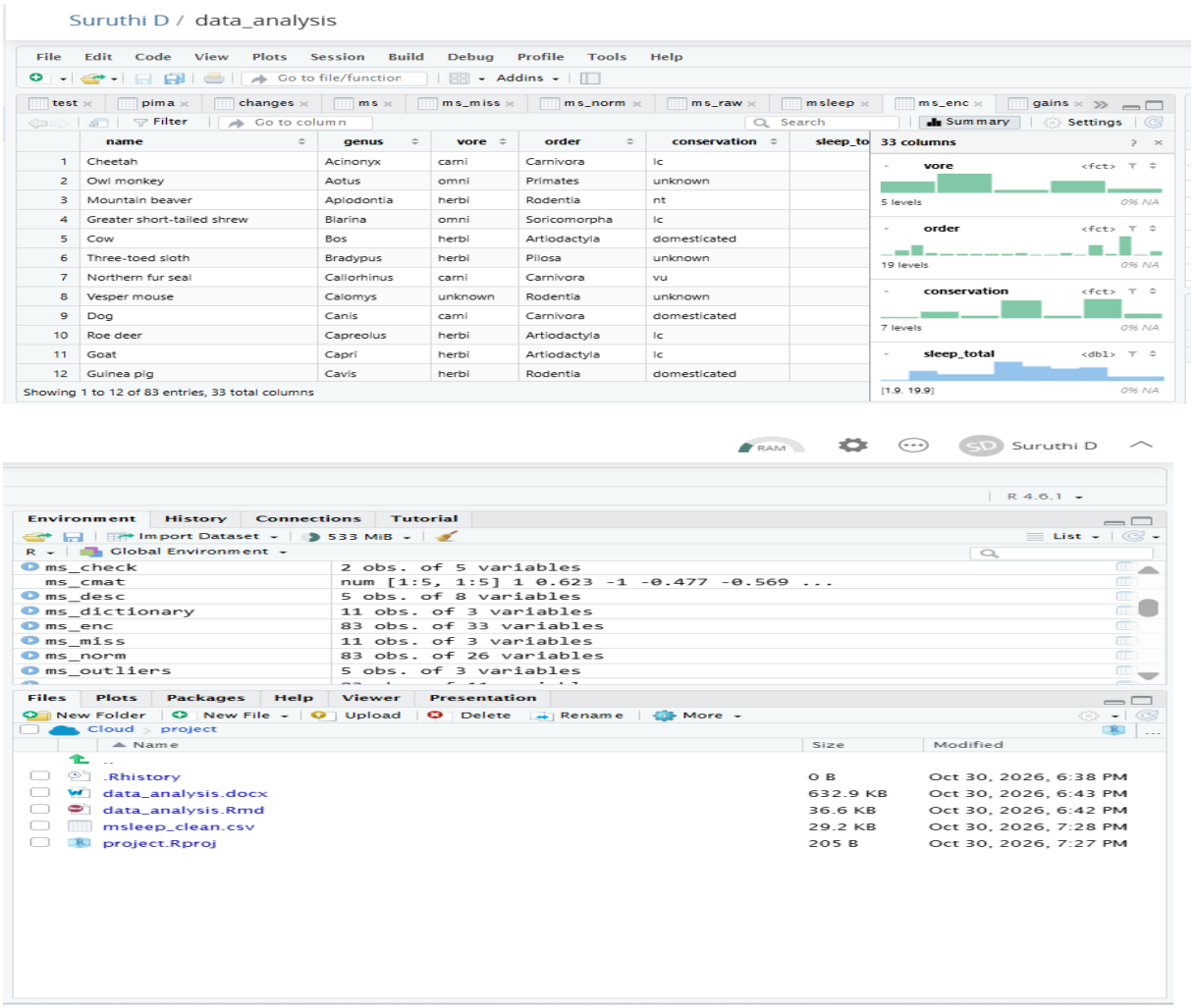


Fig: *Project running in RStudio on Posit Cloud, showing the Rmd file, data objects and knitted output.*

Appendix

A: Project Repositories

- Week 1, data cleaning: https://github.com/suruthi-1/r-data-cleaning-analysis
- Week 2, visualization: https://github.com/suruthi-1/r-data-visualization
- Week 3, predictive modeling: https://github.com/suruthi-1/r-statistical-modeling

B: Reproducibility

```
sessionInfo()
```

```
## R version 4.6.1 (2026-06-24)
## Platform: x86_64-pc-linux-gnu
## Running under: Ubuntu 24.04.5 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/openblas-pthread/libblas.so.3
## LAPACK: /usr/lib/x86_64-linux-gnu/openblas-pthread/libopenblasp-r0.3.26.so;
  LAPACK version 3.12.0
##
## locale:
##  [1] LC_CTYPE=C.UTF-8       LC_NUMERIC=C           LC_TIME=C.UTF-8
##  [4] LC_COLLATE=C.UTF-8     LC_MONETARY=C.UTF-8    LC_MESSAGES=C.UTF-8
##  [7] LC_PAPER=C.UTF-8       LC_NAME=C              LC_ADDRESS=C
## [10] LC_TELEPHONE=C         LC_MEASUREMENT=C.UTF-8 LC_IDENTIFICATION=C
##
## time zone: UTC
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] pROC_1.19.1     caret_7.0-1     lattice_0.22-9  gapminder_1.0.1
##  [5] knitr_1.52      lubridate_1.9.5 forcats_1.0.1   stringr_1.6.0
##  [9] dplyr_1.2.1     purrr_1.2.2     readr_2.2.0     tidyr_1.3.2
## [13] tibble_3.3.1    ggplot2_4.0.3   tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
##  [1] tidyselect_1.2.1     timeDate_4052.112    farver_2.1.2
##  [4] S7_0.2.2             fastmap_1.2.0        digest_0.6.39
##  [7] rpart_4.1.27         timechange_0.4.0     lifecycle_1.0.5
## [10] survival_3.8-6       magrittr_2.0.5       compiler_4.6.1
## [13] rlang_1.3.0          tools_4.6.1          yaml_2.3.12
## [16] data.table_1.18.6.1  labeling_0.4.3       bit_4.6.0
## [19] plyr_1.8.9           RColorBrewer_1.1-3   withr_3.0.3
## [22] nnet_7.3-20          grid_4.6.1           stats4_4.6.1
## [25] e1071_1.7-17         future_1.80.0        globals_0.20.0
## [28] scales_1.4.0         iterators_1.0.14     MASS_7.3-65
## [31] cli_3.6.6            rmarkdown_2.32       crayon_1.5.3
## [34] generics_0.1.4       otel_0.2.0           rstudioapi_0.19.0
## [37] future.apply_1.21.0  reshape2_1.4.5       tzdb_0.5.0
## [40] proxy_0.4-29         splines_4.6.1        parallel_4.6.1
## [43] vctrs_0.7.3          hardhat_1.4.3        Matrix_1.7-5
## [46] hms_1.1.4            bit64_4.8.6          listenv_1.1.0
```

```
## [49] foreach_1.5.2        gower_1.0.2          recipes_1.4.0
## [52] glue_1.8.1           parallelly_1.48.0    codetools_0.2-20
## [55] stringi_1.8.9        gtable_0.3.6         pillar_1.11.1
## [58] htmltools_0.5.9      ipred_0.9-16         lava_1.10
## [61] R6_2.6.1             vroom_1.7.1          evaluate_1.0.5
## [64] class_7.3-23         Rcpp_1.1.2           nlme_3.1-169
## [67] prodlim_2026.03.11   mgcv_1.9-4           xfun_0.62
## [70] pkgconfig_2.0.3      ModelMetrics_1.2.2.2
```